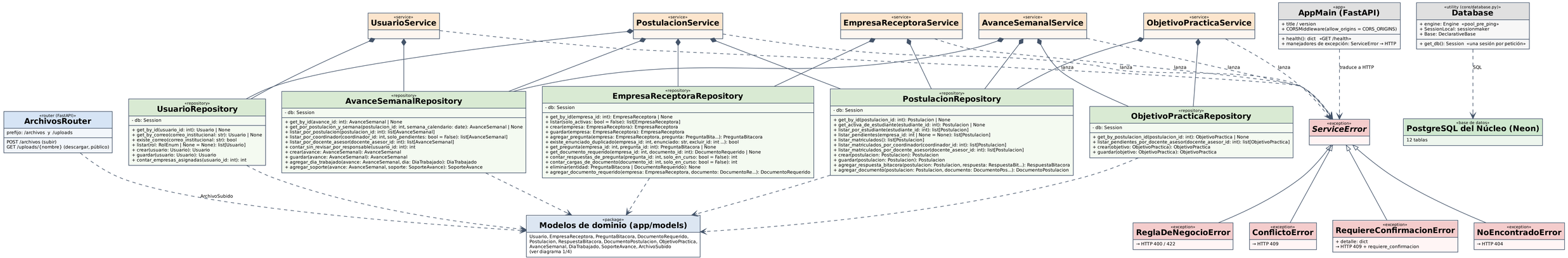
JEDANIX — Diagrama de clases (3/5): Núcleo — Servicios, Repositorios, Modelos y Excepciones
«router (FastAPI)»
ArchivosRouter
prefijo: /archivos y /uploads
POST /archivos (subir)
GET /uploads/{nombre} (descargar, público)
ArchivoSubido
«service»
UsuarioService
«service»
PostulacionService
«service»
EmpresaReceptoraService
«service»
AvanceSemanalService
«service»
ObjetivoPracticaService
lanza
lanza
lanza
lanza
lanza
«app»
AppMain (FastAPI)
+ title / version
+ CORSMiddleware(allow_origins = CORS_ORIGINS)
+ health(): dict «GET /health»
+ manejadores de excepción: ServiceError → HTTP
traduce a HTTP
«utility (core/database.py)»
Database
+ engine: Engine «pool_pre_ping»
+ SessionLocal: sessionmaker
+ Base: DeclarativeBase
+ get_db(): Session «una sesión por petición»
SQL
«repository»
UsuarioRepository
- db: Session
+ get_by_id(usuario_id: int): Usuario | None
+ get_by_correo(correo_institucional: str): Usuario | None
+ existe_correo(correo_institucional: str): bool
+ listar(rol: RolEnum | None = None): list[Usuario]
+ crear(usuario: Usuario): Usuario
+ guardar(usuario: Usuario): Usuario
+ contar_empresas_asignadas(usuario_id: int): int
«repository»
AvanceSemanalRepository
- db: Session
+ get_by_id(avance_id: int): AvanceSemanal | None
+ get_por_postulacion_y_semana(postulacion_id: int, semana_calendario: date): AvanceSemanal | None
+ listar_por_postulacion(postulacion_id: int): list[AvanceSemanal]
+ listar_por_coordinador(coordinador_id: int, solo_pendientes: bool = False): list[AvanceSemanal]
+ listar_por_docente_asesor(docente_asesor_id: int): list[AvanceSemanal]
+ contar_sin_revisar_por_responsable(usuario_id: int): int
+ crear(avance: AvanceSemanal): AvanceSemanal
+ guardar(avance: AvanceSemanal): AvanceSemanal
+ agregar_dia_trabajado(avance: AvanceSemanal, dia: DiaTrabajado): DiaTrabajado
+ agregar_soporte(avance: AvanceSemanal, soporte: SoporteAvance): SoporteAvance
«repository»
EmpresaReceptoraRepository
- db: Session
+ get_by_id(empresa_id: int): EmpresaReceptora | None
+ listar(solo_activas: bool = False): list[EmpresaReceptora]
+ crear(empresa: EmpresaReceptora): EmpresaReceptora
+ guardar(empresa: EmpresaReceptora): EmpresaReceptora
+ agregar_pregunta(empresa: EmpresaReceptora, pregunta: PreguntaBita...): PreguntaBitacora
+ existe_enunciado_duplicado(empresa_id: int, enunciado: str, excluir_id: int ...): bool
+ get_pregunta(empresa_id: int, pregunta_id: int): PreguntaBitacora | None
+ get_documento_requerido(empresa_id: int, documento_id: int): DocumentoRequerido | None
+ contar_respuestas_de_pregunta(pregunta_id: int, solo_en_curso: bool = False): int
+ contar_cargas_de_documento(documento_id: int, solo_en_curso: bool = False): int
+ eliminar(entidad: PreguntaBitacora | DocumentoRequerido): None
+ agregar_documento_requerido(empresa: EmpresaReceptora, documento: DocumentoRe...): DocumentoRequerido
«repository»
PostulacionRepository
- db: Session
+ get_by_id(postulacion_id: int): Postulacion | None
+ get_activa_de_estudiante(estudiante_id: int): Postulacion | None
+ listar_por_estudiante(estudiante_id: int): list[Postulacion]
+ listar_pendientes(empresa_id: int | None = None): list[Postulacion]
+ listar_matriculados(): list[Postulacion]
+ listar_matriculados_por_coordinador(coordinador_id: int): list[Postulacion]
+ listar_matriculados_por_docente_asesor(docente_asesor_id: int): list[Postulacion]
+ crear(postulacion: Postulacion): Postulacion
+ guardar(postulacion: Postulacion): Postulacion
+ agregar_respuesta_bitacora(postulacion: Postulacion, respuesta: RespuestaBit...): RespuestaBitacora
+ agregar_documento(postulacion: Postulacion, documento: DocumentoPos...): DocumentoPostulacion
«repository»
ObjetivoPracticaRepository
- db: Session
+ get_by_postulacion_id(postulacion_id: int): ObjetivoPractica | None
+ listar_pendientes_por_docente_asesor(docente_asesor_id: int): list[ObjetivoPractica]
+ crear(objetivo: ObjetivoPractica): ObjetivoPractica
+ guardar(objetivo: ObjetivoPractica): ObjetivoPractica
«exception»
ServiceError
«base de datos»
PostgreSQL del Núcleo (Neon)
12 tablas
«package»
Modelos de dominio (app/models)
Usuario, EmpresaReceptora, PreguntaBitacora, DocumentoRequerido, Postulacion, RespuestaBitacora, DocumentoPostulacion, ObjetivoPractica, AvanceSemanal, DiaTrabajado, SoporteAvance, ArchivoSubido
(ver diagrama 1/4)
«exception»
ReglaDeNegocioError
→ HTTP 400 / 422
«exception»
ConflictoError
→ HTTP 409
«exception»
RequiereConfirmacionError
+ detalle: dict
→ HTTP 409 + requiere_confirmacion
«exception»
NoEncontradoError
→ HTTP 404